

Universidad Mariano Gálvez de Guatemala



Ingeniero David Alvarez

Programación III

Proyecto Final

Red Social Académica con Estructuras de Datos en C++ y Benchmark

Comparativo con Java

Priscila Sarai Guzmán Calgua

9941-23-450

Introducción

El presente proyecto consiste en el desarrollo de una Red Social Académica utilizando estructuras de datos avanzadas implementadas manualmente en C++. El sistema permite administrar estudiantes y generar rankings académicos. Además, se desarrollo un módulo de benchmark en Java utilizando estructuras nativas del framework del lenguaje con el objetivo de comparar rendimiento, facilidad de implementación y nivel de abstracción respecto a las estructuras implementadas manualmente en C++.

Objetivo General

Desarrollar un sistema en C++ que simule una red social academica universitaria utilizando estructuras de datos avanzadas y compararla con implementaciones equivalentes en Java.

Objetivos Específicos

- Implementar una Tabla Hash utilizando Separate Chaining.
- Implementar un Árbol AVL para ranking académico.
- Implementar un Grafo no dirigido para conexiones entre estudiantes.
- Implementar una Lista Enlazada para proyectos académicos.
- Implementar una Pila para historial de acciones.
- Comparar el rendimiento entre C++ y Java mediante benchmarks.
- Incorporar una mejora avanzada mediante el algoritmo de ruta más corta entre estudiantes.

Descripción del Sistema

El sistema representa una red social académica donde los estudiantes pueden:

- Registrarse dentro de la plataforma
- Ser localizados rápidamente mediante su ID
- Agregar proyectos académicos
- Conectarse con otros estudiantes
- Ranking académico
- Analizar el rendimiento de las estructuras utilizadas

```
"C:\Users\prisc\OneDrive\Doc  X  +  v

=====
Priscila Sarai Guzmán Calgua 9941-23-450
=====

MENU PRINCIPAL
=====
1. Cargar estudiantes desde CSV
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto a estudiante
6. Mostrar proyectos de estudiante
7. Buscar proyecto de estudiante
8. Eliminar proyecto de estudiante
9. Conectar estudiantes
10. Verificar si dos estudiantes estan conectados
11. Mostrar conexiones de estudiante
12. Recorrer red con BFS
13. Recorrer red con DFS
14. Mostrar ranking academico
15. Mostrar estadisticas de estructuras
16. Generar benchmark C++
17. Ruta mas corta entre estudiantes
18. Ver historial de acciones
19. Salir
=====
Seleccione una opcion (1-19): 1

Cargando base de datos desde 'estudiantes.csv'...
>>> Se cargaron 10000 estudiantes exitosamente.

Tiempo de carga: 204.885 ms
=====
```

Estructuras de Datos Implementadas

Tabla Hash: se utiliza para almacenar estudiantes mediante su ID.

Funciones implementadas:

- Insertar estudiante

```
=====
Seleccione una opcion (1-19): 2

--- Registrar Nuevo Estudiante ---
Ingrese ID: 11001
Ingrese Nombre Completo: Priscila Guzmán
Ingrese Carrera: Ingenieria en Sistemas
Ingrese Semestre actual: 5
Ingrese Promedio (GPA): 85
Ingrese Puntaje de Ranking (Skill Score): 250

Exitó Estudiante registrado correctamente en todas las estructuras.
=====
```

- Buscar estudiante

```
=====
Seleccione una opcion (1-19): 3

Ingrese el ID del estudiante a buscar: 1090

>>> ESTUDIANTE ENCONTRADO <<<
ID: 1090
Nombre: Fernando Solorzano Estrada
Carrera: Ingenieria Administrativa
Semestre: 2
GPA: 77.79
Skill Score: 184
El estudiante no tiene proyectos registrados.
=====
```

- Eliminar estudiante

```
=====
Seleccione una opcion (1-19): 4

Ingrese el ID del estudiante a eliminar del sistema: 11001

Exitó El estudiante Priscila Guzmán ha sido removido de todas las estructuras.
=====
```

- Reportar colisiones
- Calcular factor de carga

Se implemento resolución de colisiones mediante Separate Chaining utilizando listas enlazadas.

Lista Enlazada de Proyectos: cada estudiante posee una lista enlazada de proyectos.

Cada proyecto almacena:

- project_id
- title
- description
- completion_year

Funciones Implementadas:

- Agregar proyecto

```
"C:\Users\prisc\OneDrive\Doc  X  +  v
Seleccione una opcion (1-19): 5

Ingrese el ID del estudiante para agregar un proyecto: 1999
Ingrese ID unico del proyecto: 10
Titulo del proyecto: Sistema Bancario
Descripcion breve: Realizar un sistemas tipo cajero
Año de finalizacion: 2026

Exito Proyecto incorporado a la Lista Enlazada del estudiante.
=====

=====
Seleccione una opcion (1-19): 5

Ingrese el ID del estudiante para agregar un proyecto: 1999
Ingrese ID unico del proyecto: 12
Titulo del proyecto: facturas
Descripcion breve: creacion de sistemas de facturacion para una farmacia
Año de finalizacion: 2027

Exito Proyecto incorporado a la Lista Enlazada del estudiante.
```

- Mostrar proyecto

```
=====
Seleccione una opcion (1-19): 6

Ingrese el ID del estudiante para ver sus proyectos: 1001
El estudiante no tiene proyectos registrados.
=====
```

- Buscar proyecto

```
=====
Seleccione una opcion (1-19): 7

Ingrese ID del estudiante: 1999
Ingrese ID del proyecto: 12

=== PROYECTO ENCONTRADO ===
ID: 12
Titulo: facturas
Descripcion: creacion de sistemas de facturacion para una farmacia
Anio: 2027
=====
```

- Eliminar proyecto

```
=====
Seleccione una opcion (1-19): 8

Ingrese ID del estudiante: 1999
Ingrese ID del proyecto a eliminar: 12

Proyecto eliminado correctamente.
=====
```

- Se verifico que ya no exista el proyecto que se acaba de eliminar

```
=====
Seleccione una opcion (1-19): 6

Ingrese el ID del estudiante para ver sus proyectos: 1999
--- Proyectos de Paola Reyes Ajin ---
ID: 10 | Titulo: Sistema Bancario
    Descripcion: Realizar un sistemas tipo cajero
    Ano de Finalizacion: 2026
=====
```

Grafo no Dirigido: el grafo representa las conexiones entre estudiantes.

Funciones implementadas:

- Conectar estudiantes

```
=====
Seleccione una opcion (1-19): 9

--- Establecer Conexion ---
Ingrese el ID del primer estudiante: 1001
Ingrese el ID del segundo estudiante: 1002
[EXITO] Conexion creada exitosamente entre 1001 y 1002.
=====
```

```
=====
Seleccione una opcion (1-19): 9
=====
```

```
--- Establecer Conexion ---
```

```
Ingrese el ID del primer estudiante: 1001
```

```
Ingrese el ID del segundo estudiante: 1003
```

```
[EXITO] Conexion creada exitosamente entre 1001 y 1003.
=====
```

```
=====
Seleccione una opcion (1-19): 9
=====
```

```
--- Establecer Conexion ---
```

```
Ingrese el ID del primer estudiante: 1002
```

```
Ingrese el ID del segundo estudiante: 1004
```

```
[EXITO] Conexion creada exitosamente entre 1002 y 1004.
=====
```

- Verificar conectividad

```
=====
Seleccione una opcion (1-19): 10
=====
```

```
--- Verificar Conectividad Directa ---
```

```
Ingrese ID del primer estudiante: 1002
```

```
Ingrese ID del segundo estudiante: 1003
```

```
Resultado:NO existe una conexion directa entre los estudiantes consultados.
=====
```

```
=====
Seleccione una opcion (1-19): 10
=====
```

```
--- Verificar Conectividad Directa ---
```

```
Ingrese ID del primer estudiante: 1002
```

```
Ingrese ID del segundo estudiante: 1004
```

```
Resultado:SI existe una conexion directa entre el estudiante 1002 y el estudiante 1004.
=====
```

- Mostrar conexiones

```
=====
Seleccione una opcion (1-19): 11
=====
```

```
Ingrese el ID del estudiante: 1002
```

```
Conexiones del estudiante [1002]: 1004 1001
=====
```

```
=====
Seleccione una opcion (1-19): 11
=====
```

```
Ingrese el ID del estudiante: 1001
```

```
Conexiones del estudiante [1001]: 1003 1002
=====
```

- BFS

```
=====
Seleccione una opcion (1-19): 12

Ingrese el ID del estudiante para iniciar el recorrido BFS: 1002

--- Recorrido BFS (Amplitud) ---
Recorrido BFS desde ID 1002: 1002 1004 1001 1003 1005 1006
```

- DFS

```
=====
Seleccione una opcion (1-19): 13

Ingrese el ID del estudiante para iniciar el recorrido DFS: 1002

--- Recorrido DFS (Profundidad) ---
Recorrido DFS desde ID 1002: 1002 1004 1003 1005 1006 1001
```

Mejora Implementada

Ruta más corta entre estudiantes: Como funcionalidad adicional se implementó un algoritmo de búsqueda de ruta más corta utilizando BFS sobre el grafo social. La funcionalidad permite determinar la secuencia mínima de conexiones necesarias para llegar desde un estudiante origen hasta un estudiante destino. Ejemplo:

```
"C:\Users\prisc\OneDrive\Doc  X + v
=====
Seleccione una opcion (1-19): 17

===== RUTA MAS CORTA =====
Ingrese ID de origen: 1003
Ingrese ID destino: 1002

===== RUTA MAS CORTA =====
1003 -> 1004 -> 1002
Longitud de la ruta: 2 conexiones.
=====
```

```
=====
Seleccione una opcion (1-19): 17

===== RUTA MAS CORTA =====
Ingrese ID de origen: 1002
Ingrese ID destino: 1005

===== RUTA MAS CORTA =====
1002 -> 1004 -> 1003 -> 1005
Longitud de la ruta: 3 conexiones.
```


Árbol AVL: se utiliza para administrar el ranking académico basado en el atributo skill core.

Funciones implementadas:

- Inserción balanceada
- Recorrido InOrden
- Ranking académico
- El que tiene score más bajo ahora es el ultimo

9963	3283	Katherine Ortiz Mendoza	Score: 141
9964	3086	Alejandra Sanchez Cruz	Score: 141
9965	10789	Paola Aragon Pineda	Score: 140
9966	6215	Jose Diaz Hernandez	Score: 140
9967	5604	Susana Navarro Gomez	Score: 140
9968	3908	Raul Estrada Garcia	Score: 140
9969	8505	Victor Quinteros Leon	Score: 139
9970	8192	Mario Aguilar Hernandez	Score: 139
9971	7849	Pedro Rodriguez Chavez	Score: 139
9972	6539	Monica Villatoro Aragon	Score: 139
9973	6386	Bryan Alvarez Rodriguez	Score: 139
9974	5792	Mario Villatoro Lopez	Score: 139
9975	10443	Roberto Ajin Reyes	Score: 138
9976	7304	Mariana Castillo Barrios	Score: 138
9977	2065	Laura Ortiz Cruz	Score: 136
9978	1274	Alejandra Alvarado Leon	Score: 136
9979	8687	Marta Reyes Diaz	Score: 135
9980	5533	Angel Reyes Chavez	Score: 135
9981	4163	Elena Alvarado Rivas	Score: 135
9982	3345	Andres Delgado Mejia	Score: 135
9983	10558	Lucia Barrios Rodriguez	Score: 134
9984	2537	Andrea Mendoza Cabrera	Score: 134
9985	1803	Cristian Gonzalez Cifuentes	Score: 134
9986	6314	Alejandra Rosales Escobar	Score: 133
9987	10073	Elena Gomez Perez	Score: 131
9988	8392	Juan Diaz Rosales	Score: 131
9989	3627	Samuel Garcia Chavez	Score: 131
9990	2365	Marta Quinteros Morales	Score: 131
9991	1409	Angel Morales Diaz	Score: 131
9992	3957	Valeria Castro Ortiz	Score: 130
9993	1684	Erick Ramirez Castillo	Score: 129
9994	9186	Raul Torres Flores	Score: 128
9995	3500	Daniela Cabrera Mendez	Score: 122
9996	10389	Camila Fuentes Sanchez	Score: 121
9997	10821	Miguel Sanchez Cifuentes	Score: 119
9998	6140	Melissa Lopez Escobar	Score: 119
9999	7541	Fabiola Cabrera Velasquez	Score: 109
10000	4989	Elena Vasquez Quinteros	Score: 105

Estadísticas del Sistema

- Total de estudiantes cargados
- Total de proyectos registrados
- Total de conexiones creadas
- Total de colisiones
- Factor de carga
- Altura del AVL
- Tiempo total de carga del dataset

```
=====
Seleccione una opcion (1-19): 15
=====
                        ESTADISTICAS ESTRUCTURALES
=====

[Estadisticas Tabla Hash]
- Total estudiantes: 10000
- Colisiones totales: 8001
- Factor de carga: 5

[Estadisticas AVL]
- Altura actual del arbol: 15
- Total estudiantes registrados: 10000
Total de proyectos registrados: 1

[Estadisticas Grafo Social]
=====
                        ESTADISTICAS DEL GRAFO
=====
Total de Estudiantes (Vertices): 10001
Total de Conexiones Reales (Aristas): 6
Total de Conexiones en Memoria (Punteros): 12
Conexiones promedio por estudiante: 0.00119988
=====

Total de conexiones creadas: 6

Tiempo total de carga del dataset: 204.885 ms
=====
```

Pila de Historial: se implemento una pila para almacenar las acciones realizadas por el usuario dentro del sistema.

```
=====
Seleccione una opcion (1-19): 18

=====
                HISTORIAL DE ACCIONES DEL SISTEMA
=====

--- HISTORIAL DE ULTIMAS ACCIONES (18 en total) ---
[1] Se ejecuto el modulo de Benchmark de C++. Archivo cpp_results.csv generado.
[2] Visualizacion completa del Ranking Academico Global
[3] Consulta de ruta mas corta entre ID 1002 y ID 1005
[4] Consulta de ruta mas corta entre ID 1003 y ID 1002
[5] Conexion social registrada entre ID 1005 e ID 1006.
=====
```

```
=====
Seleccione una opcion (1-19): 19

Cerrando el sistema de Red. Liberando recursos de memoria asignados de forma dinamica...

Process returned -1073740940 (0xC0000374)   execution time : 1042.206 s
Press any key to continue.
|
```

Benchmark en C++: fue ejecutado sobre estructuras limpias independientes para evitar contaminación de resultados, se midió:

- Inserción en TablaHash
- Búsqueda en TablaHash
- Inserción en AVL
- Recorrido AVL
- BFS en Grafo
- DFS en Grafo

Estos resultados se generan automáticamente en un archivo cpp_results.csv

The screenshot shows a C++ benchmark application running in a terminal window. The application has a menu with 19 options. Option 16, 'Ejecutar Benchmark aislado con 10,000 registros...', is selected. The benchmark results are displayed in the terminal, showing the execution of various operations on a graph with 10,000 records. The results are then saved to a CSV file named 'cpp_results.csv'.

```
2. Registrar estudiante manualmente
3. Buscar estudiante por ID
4. Eliminar estudiante
5. Agregar proyecto a estudiante
6. Mostrar proyectos de estudiante
7. Buscar proyecto de estudiante
8. Eliminar proyecto de estudiante
9. Conectar estudiantes
10. Verificar si dos estudiantes estan conectados
11. Mostrar conexiones de estudiante
12. Recorrer red con BFS
13. Recorrer red con DFS
14. Mostrar ranking academico
15. Mostrar estadísticas de estructuras
16. Generar benchmark C++
17. Ruta mas corta entre estudiantes
18. Ver historial de acciones
19. Salir
=====
Seleccione una opcion (1-19): 16

Ejecutando Benchmark aislado con 10,000 registros...

=====
INICIANDO BENCHMARK DE RENDIMIENTO C++
=====
-> Midiendo insercion en Tabla Hash (10000 registros)...
-> Midiendo busqueda en Tabla Hash (10000 operaciones)...
-> Midiendo insercion en Arbol AVL (10000 registros)...
-> Midiendo recorrido InOrden en Arbol AVL...
-> Inicializando nodos y aristas en el Grafo...
-> Midiendo recorrido BFS en Grafo...
-> Midiendo recorrido DFS en Grafo...

[!] BENCHMARK FINALIZADO CON EXITO.
>>> Se ha generado el archivo 'cpp_results.csv' con todas las metricas solicitadas.

Modulo completado. Archivo 'cpp_results.csv' generado de forma limpia.
```

The Excel spreadsheet shows the results of the benchmark. The first row contains the headers: language, operation, structure, records, time_ms. The subsequent rows show the results for various operations on a graph with 10,000 records.

language	operation	structure	records	time_ms
C++	insert	HashTable	10000	0
C++	search	HashTable	10000	0
C++	insert	AVL	10000	15.797
C++	traversal	AVL	10000	0
C++	bfs	Graph	10000	475.404
C++	dfs	Graph	10000	316.934

Benchmark en Java: se desarrollo un benchmark equivalente utilizando estructuras nativas del framework Java.

Estructuras utilizadas:

- HashMap
- TreeMap

Los resultados se generan en un archivo llamado java_results.csv

The screenshot shows a Java benchmark application running in an IDE console. The application has a menu with 19 options. Option 16, 'Ejecutar Benchmark aislado con 10,000 registros...', is selected. The benchmark results are displayed in the console, showing the execution of various operations on a graph with 10,000 records. The results are then saved to a CSV file named 'java_results.csv'.

```
<terminated> BenchmarkMain [Java Application] C:\Users\prisc\OneDrive\Documents\BenchmarkMain\BenchmarkMain.java
===== EJECUTANDO BENCHMARK EN JAVA =====
[Éxito] Archivo java_results.csv
=====
Benchmark finalizado de manera correcta.
Tiempo total de carga: 698.1578 ms
```

A1	language,operation,structure,records,time_ms
2	java,insert,HashMap,10000,289.8671
3	java,insert,TreeMap,10000,266.1550
4	java,insert,Grafo,10000,89.2651
5	java,search,HashMap,10000,0.0033
6	java,search,TreeMap,10000,0.1415
7	java,search,Grafo,10000,0.0020
8	java,traversal,TreeMap,10000,7.9569
9	java,traversal,Grafo,10000,44.2151

Tabla comparativa de Benchmarks

Operación	Estructura C++	Tiempo C++ (ms)	Estructura Java	Tiempo Java (ms)
Inserción	HashTable	0.000	HashMap	289.8671
Búsqueda	HashTable	0.000	HashMap	0.0033
Inserción	AVL	15.797	TreeMap	266.1550
Recorrido	AVL	0.000	TreeMap	7.9569
BFS	Graph	475.404	Grafo	44.2151
DFS	Graph	316.934	Grafo	44.2151*

Se realizaron pruebas de rendimiento utilizando 10,000 registros tanto en el sistema desarrollado en C++ como en el benchmark implementado en Java. Los resultados muestran que las operaciones de búsqueda en ambas tecnologías presentan tiempos extremadamente bajos, debido a la eficiencia de las estructuras utilizadas. En el caso de C++, la Tabla Hash implementada manualmente logró tiempos prácticamente instantáneos para inserciones y búsquedas. Para las estructuras de tipo árbol, el Árbol AVL desarrollado en C++ obtuvo mejores tiempos de inserción que TreeMap en Java. Esto se debe a que la implementación

en C++ trabaja directamente sobre memoria y posee menor sobrecarga de abstracción.

Respecto a los recorridos de grafos, los resultados muestran diferencias importantes entre ambas implementaciones. Estas diferencias pueden estar influenciadas por la forma en que se construyó el grafo, la cantidad de conexiones generadas y el entorno de ejecución utilizado para cada lenguaje.

En términos generales, C++ ofrece mayor control sobre la memoria y las estructuras internas, permitiendo optimizaciones de bajo nivel. Por otro lado, Java proporciona estructuras nativas altamente optimizadas que simplifican considerablemente el desarrollo y reducen la complejidad de implementación.

La comparación permitió comprender las ventajas y desventajas de implementar estructuras manualmente frente al uso de colecciones ya integradas en un framework moderno.

Conclusiones

- La Tabla Hash permitió búsquedas eficientes en tiempo cercano a $O(1)$.
- El Árbol AVL mantuvo el balance automáticamente garantizando búsquedas e inserciones eficientes.
- El Grafo permitió modelar correctamente las relaciones académicas entre estudiantes y ejecutar recorridos BFS y DFS.
- Java simplifica considerablemente la implementación mediante estructuras ya optimizadas como HashMap y TreeMap.
- C++ ofrece mayor control sobre memoria y estructuras internas, mientras que Java proporciona una capa de abstracción que acelera el desarrollo.
- La mejora avanzada de ruta más corta incrementó la funcionalidad del sistema permitiendo analizar relaciones indirectas entre estudiantes.

[Video](#)

[Repositorio de GitHub](#)